

Demo Script — Village of Garden City AI Receptionist

Everything you need to run the demo, plus what to say and what to expect.

Before you start (do this 15 minutes early)

1. Free up the machine — this matters most

Local AI inference competes with everything else for CPU and RAM. During testing, VS Code alone consumed 276% CPU and pushed response times from ~2s to ~10s.

Quit before demoing:

- VS Code / any IDE
- Extra Chrome windows (keep one, with two tabs)
- Slack, Spotify, Docker Desktop, Zoom
- Anything doing background sync

```
# Check you have headroom — want 4GB+ free
vm_stat | awk '/free/{f=$3} /inactive/{i=$3} END {gsub(/\
```

Also: **plug in the power adapter**. macOS throttles CPU aggressively on battery.

2. Start the three services

```
# 1. Ollama (usually already running)
brew services start ollama

# 2. Backend
cd "~/Desktop/~JAR_VIS~/~AI PROJECT~/CallCenter"
./scripts/dev.sh

# 3. Tunnel – only if demoing the public URL
./scripts/tunnel.sh run
```

3. Warm everything up

Cold model loads cost 5–10 seconds. Do this before anyone is watching:

```
curl -s -X POST http://127.0.0.1:8000/api/voice/synthesize \
  -H 'Content-Type: application/json' -d '{"text": "Warmin"}'

curl -s -X POST http://127.0.0.1:8000/api/chat \
  -H 'Content-Type: application/json' \
  -d '{"message": "When is garbage collection?"}' > /dev/null

echo "warm"
```

Then **run one full voice turn yourself** in the browser. The first transcription loads Whisper (~5s once).

4. Confirm green

```
curl -s http://127.0.0.1:8000/api/health | python3 -m json.tool
```

Want `"ready_for_calls": true` and `llm, embedding, vector_store` all ok.

The demo, in order

Roughly 8 minutes. Each scenario proves a different thing.

Opening line

"This is a working prototype of an AI receptionist for the Village. Everything you're about to see runs on this laptop — no cloud service, no per-call cost, and no resident data leaving the machine. It answers using the Village's own published information, and it's built to refuse rather than guess."

That last clause is the whole pitch. Say it up front.

Scenario 1 — It answers from real Village content

Say: *"When is garbage collection?"*

Expect: Routes to Sanitation, answers "each Wednesday, except as noted on the Holiday Schedule," with a source card linking to gardencityny.net.

Point out: "That answer came from the Village's own Sanitation page. Click the source — it's the live page. Every answer carries its citation, so staff can verify anything it says."

Scenario 2 — It understands follow-ups

Say: *"I have a question about garbage collection."* **Expect:** *"Sure, I can help with that. That's handled by Sanitation. What would you like to know?"* — it asks rather than guessing.

Then say: *"When is mine?"* **Expect:** Correct collection answer. "Mine" resolved from context.

Point out: "It remembered we were talking about sanitation. That context lasts for the call and is discarded afterwards — there's no caller profile."

Scenario 3 — It routes correctly

Say: *"I need a building permit."* **Expect:** Building Department, with location detail.

Say: *"How do I pay my water bill?"* **Expect:** Finance, with the payment method from the Village site.

Point out: "Nine departments. Most routing happens on deterministic rules — no AI call at all — which makes it fast and auditable. The model only gets involved when a request is genuinely ambiguous."

Scenario 4 — ☆ It refuses to guess (the most important one)

Say: *"Can I keep chickens in my backyard?"*

Expect: It does **not** answer. Either a clarifying question or an escalation offer.

Point out — this is your headline:

"This is the part that matters. The Village website doesn't cover this, so it won't invent an answer. Most AI assistants would produce something plausible and wrong. A resident acting on a made-up permit rule is a real problem for the Village — so the system is designed to hand off instead."

Then show /admin/review: the question is queued for staff with the evidence it tried and why it declined.

Scenario 5 — Safety overrides

Say: *"Can I sue the Village over a sidewalk?"* **Expect:** Immediate refusal — legal matters always go to a person. Under 2s, because it never reaches the AI at all.

Say: *"There's a gas leak on my street."* **Expect:** *"If this is an emergency, please hang up and dial 911 right away."*

Point out: "Legal questions, account lookups, and emergencies are hard-coded to escalate. No confidence score can override that. Emergencies never touch the AI."

Scenario 6 — Staff can teach it

1. Open `/admin/review` — show the queued chicken question
2. Write an answer, pick the department, click **Approve**
3. Return to the receptionist and ask the same question — it now answers

Point out:

"The AI can never add to its own knowledge. Only a person can, through this screen. When staff answer a question once, every future caller gets it. It improves through your team, not on its own."

Scenario 7 — The dashboard

Open `/admin`.

Point out: conversations, AI-resolved rate, escalations, response times, department volume, and the confidence distribution — all from real calls, none hardcoded. "This tells you which departments are getting the most calls and what residents ask that you haven't documented."

Quick reference — expected results

You say	Department	Behavior
"When is garbage collection?"	Sanitation	Answers, cited
"Where do I report a pothole?"	Public Works	Answers or clarifies
"I need a building permit"	Building	Answers, cited
"How do I pay my water bill?"	Finance	Answers, cited
"Railroad parking permit?"	Parking	Answers with fees
"My street light is out"	Public Works	Answers, cited
"Can I keep chickens?"	Building	Declines / clarifies
"Can I sue the Village?"	—	Escalates, ~1.5s
"Gas leak emergency"	—	911 notice
"Hello" / "Thank you"	—	Conversational, instant

If something goes wrong

Stay calm and narrate it. A prototype behaving imperfectly is expected; how you handle it is what they're judging.

Problem	Do this	Say this
Slow response (>10s)	Keep going	"Running on this laptop — a hosted model responds in under a second."
Mic won't work	Type the question — same pipeline	"Let me type this one."

Wrong/odd answer	Show <code>/admin/review</code>	"That's exactly what this queue is for."
Backend crashed	<code>./scripts/dev.sh</code>	"One moment, restarting."
Total failure	Open <code>/demo</code>	Scripted scenarios, no mic needed

Always have `/demo` open in a second tab as your safety net.

Things to say that land well

- "It runs entirely on this laptop. No cloud, no per-call cost, no resident data leaving the building."
- "Every answer cites the Village page it came from."
- "It refuses rather than guesses. That's a design decision, not a limitation."
- "It never adds to its own knowledge — only your staff can."
- "It knows nothing except what's on your website today. Everything it doesn't know is a gap we can see and close."

Things to avoid saying

- ✗ "It's ready to deploy" — it isn't; no authentication, unverified content
- ✗ "It's 100% accurate" — nothing is
- ✗ "It'll replace staff" — it routes to staff; say that instead
- ✗ Guessing at cost or timeline on the spot — take it away and follow up